WikiLLM Based Intelligent Knowledge Management & Question Answering System

Presented By :
Jayashan Manodya
www.jayashan.online

21 July 2026

List of Contents

Why WikiLLM instead of RAG

Traditional RAG retrieves relevant document chunks using embeddings and vector search. While effective for document retrieval, it relies heavily on semantic similarity, stores knowledge as isolated text chunks, and struggles with complex multi-step reasoning because relationships between concepts are not explicitly represented.

WikiLLM takes a different approach by converting documents into a structured, interconnected knowledge base. It automatically extracts entities, concepts, facts, definitions, relationships, and summaries, organizing them into linked wiki-style pages. Instead of retrieving disconnected text chunks, WikiLLM retrieves related knowledge pages and their connections, enabling better context understanding, improved reasoning, reduced retrieval redundancy, and more accurate answers to complex queries.

Traditional RAG	Proposed WikiLLM
Retrieves document chunks	Builds structured knowledge
Depends on vector similarity	Uses entity relationships
Limited reasoning	Better multi-hop reasoning
Repeated retrieval	Reusable knowledge base

Project Overview

This project proposes the development of an intelligent document understanding and questionanswering system using the WikiLLM architecture. Unlike traditional Retrieval-Augmented Generation (RAG) systems that rely primarily on vector databases and semantic similarity search, this system transforms uploaded documents into a structured, interconnected knowledge base before enabling question answering.

The system will automatically analyze uploaded documents, extract entities, concepts, relationships, and factual information, and organize them into a wiki-like knowledge repository. Users will then be able to ask natural language questions, and the system will generate accurate answers based on the structured knowledge.

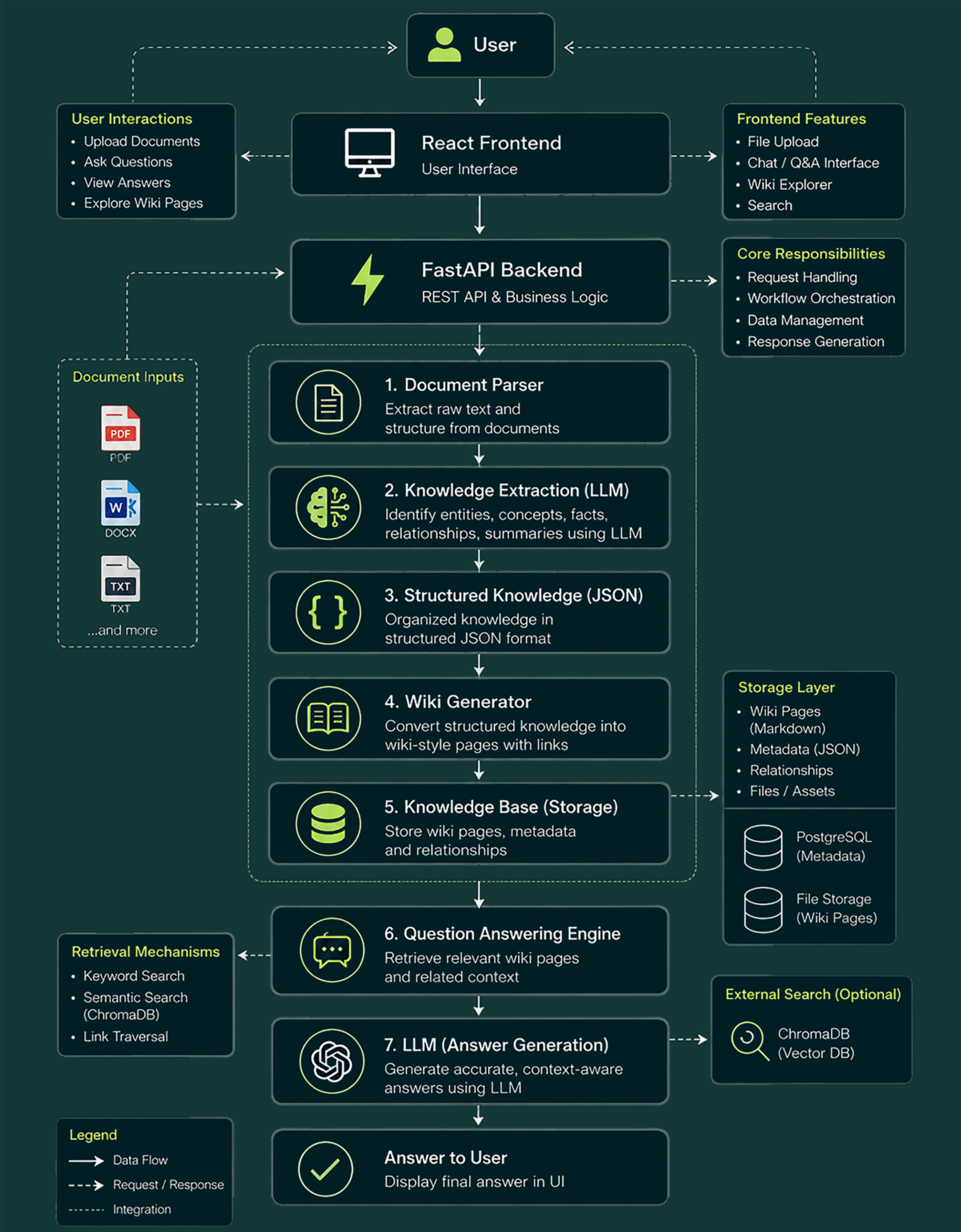
The objective is to demonstrate how Large Language Models can be used not only for retrieval but also for knowledge organization and intelligent reasoning

Objectives

The primary objectives of this project are:

- Develop a document ingestion pipeline.
- Extract structured knowledge from uploaded documents.
- Generate interconnected Wiki pages automatically.
- Store knowledge in an organized, reusable format.
- Enable natural language question answering over the generated knowledge.
- Build a scalable architecture that can support multiple documents in the future.

Proposed System Architecture



Proposed Workflow

- **Step 1 - Document Upload**

Users upload one or more documents (PDF, DOCX, or TXT).

- **Step 2 - Document Parsing**

The system extracts readable text from the uploaded document while preserving document structure where possible.

- **Step 3 - Knowledge Extraction**

Using an LLM, the system identifies:
    - Important entities
    - Key concepts Definition
    - Facts
    - Relationships between entities
    - Important summaries

The extracted knowledge will be converted into structured JSON.

Proposed Workflow

- **Step 4 - Wiki Generation**

The extracted knowledge is automatically converted into wiki-style pages.

Example: Tesla.md / Overview / Founder / CEO / Products / Related Topics / [[Electric Vehicles]] / [[Battery Technology]]

(Each page will contain references to related topics to create an interconnected knowledge network.)

- **Step 5 - Knowledge Storage**

Generated wiki pages will be stored in a structured repository.
Possible formats include:

- Markdown
- JSON
- SQLite metadata

Proposed Workflow

- Step 6 - Question Answering

When a user submits a question:

1. Relevant wiki pages are identified.
2. Related linked pages are collected.
3. Context is sent to the LLM.
4. The LLM generates the final answer

Key Features

- Document Upload
- Automatic Knowledge Extraction
- Entity Recognition
- Relationship Detection
- Wiki Page Generation
- Linked Knowledge Pages
- Natural Language Question Answering
- Multi-document Support
- Persistent Knowledge Base
- Easily Extendable Architecture

Proposed Technology Stack

Component	Component Technology
Programming language	Python
Backend	FastAPI
FrontEnd	React
Llm framework	• LangGraph (workflow orchestration) • LangChain (document processing)
Llm	GPT-4o mini / GPT-5 mini (depending on availability)
Document parsing	PyMuPDF
Knowledge storage	Markdown + JSON
Metadata storage	postgresql
Optional semantic search	ChromaDB
Version control	Git

Deliverables

The completed project will include:

- Web interface
- Source Code
- Project Documentation
- System Architecture Diagram
- Sample Knowledge Base
- Demonstration with Sample Documents
- Question Answering Demo
- API Documentation
- Deployment Guide
- Performance Evaluation

- Thank You -